

Rapport technique — ImmoSN

Audit technique du projet **ImmoSN** (plateforme SaaS immobilière, Sénégal). Document de soutenance. Toutes les affirmations sont sourcées sur le code existant ; les fichiers concernés sont cités entre parenthèses.

1. Présentation générale du projet

Objectif de l'application. ImmoSN est une plateforme web de mise en relation immobilière : publication d'annonces (vente/location), demandes de visite, suivi des prospects (leads), gestion de contrats et messagerie entre clients et administration. L'`artifactId` Maven est `sn.immosn:backend` (`backend/pom.xml`) et le domaine de production est `immosn.cloud` (`nginx/nginx.vps.conf`).

Problématique résolue. Centraliser, dans une seule application, le cycle de vie commercial d'un bien immobilier : de la consultation publique d'une annonce → demande de visite → qualification en lead → conversion en contrat → suivi du contrat (résiliation, prolongation, expiration). Chaque étape est tracée par un historique dédié (`ContratHistory`, `LeadHistory`, `VisiteHistory`).

Fonctionnalités principales (un module backend + une ou plusieurs vues frontend par fonctionnalité) :

Domaine	Backend (package <code>sn.immosn.backend</code>)	Frontend (<code>frontend/src/views</code>)
Authentification / comptes	<code>auth</code> , <code>client.web.auth</code>	<code>auth/</code> , <code>profil/</code> , <code>admin/Administrateurs*</code> , <code>admin/*Admin*</code>
Annonces, types de bien, commodités	<code>annonce</code>	<code>annonces/</code> , <code>admin/Annonces*</code> , <code>admin/TypesBien*</code> , <code>admin/Commodites*</code>
Demandes de visite	<code>visite</code>	<code>visites/</code> , <code>admin/Visites*</code>
Leads (prospects)	<code>lead</code>	<code>admin/Leads*</code>
Contrats	<code>contrat</code>	<code>contrats/</code> , <code>admin/Contrats*</code>
Discussions / messagerie	<code>discussion</code>	<code>discussions/</code> , <code>admin/Discussions*</code> , <code>admin/Messages*</code>
Signalements	<code>signalement</code>	<code>signalements/</code> , <code>admin/Signalements*</code>
Favoris	<code>favoris</code>	<code>favoris/</code>
Tableau de bord admin	<code>dashboard</code>	<code>admin/Dashboard*</code>
Géolocalisation (cartes)	<code>location</code>	<code>components/LocationMap.vue</code>

Utilisateurs et rôles. Trois rôles, définis par l'énumération `RoleType` et appliqués partout (`SecurityConfig`, guards frontend) :

- **CLIENT** — consultation d'annonces, favoris, discussions, demandes de visite, contrats propres.
- **ADMIN** — gestion des annonces, visites, leads, contrats, signalements, messages.

- **SUPER_ADMIN** — tous les droits ADMIN **plus** la gestion des comptes administrateurs (création, modification, archivage/révocation).

Cette hiérarchie est documentée directement dans le code ([config/SecurityConfig.java](#)) et un compte SUPER_ADMIN de bootstrap est créé au démarrage ([auth/service/RoleDataInitializer.java](#)).

2. Architecture globale

Type d'architecture. Application **client-serveur** en **monolithe modulaire** :

- un **backend monolithique** Spring Boot exposant une API REST stateless ;
- un **frontend SPA** (Single Page Application) Vue.js servi statiquement ;
- une **base de données** PostgreSQL managée (Neon) ;
- un **reverse proxy Nginx** qui sert le frontend et relaie `/api/` vers le backend.

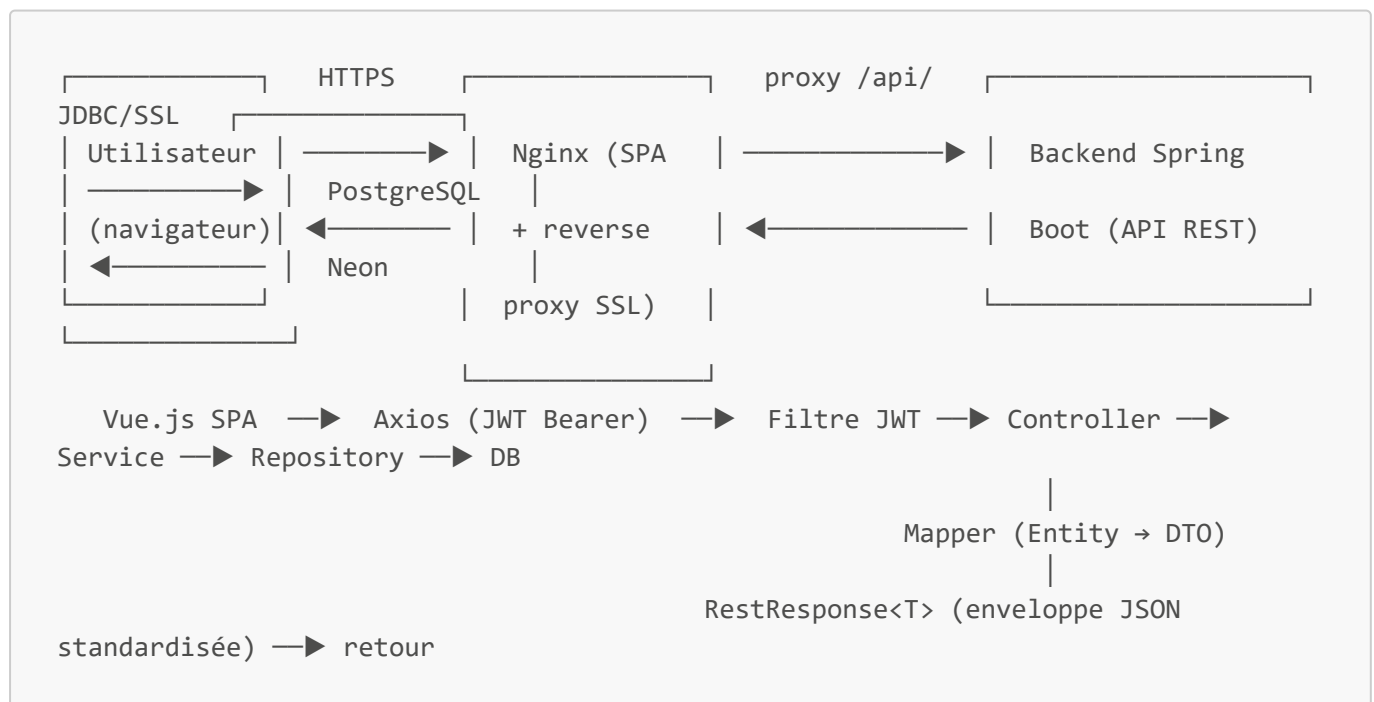
Le backend est un **monolithe organisé en modules métier** (un package racine par domaine : **annonce**, **contrat**, **visite**, **lead**, etc.), structure proche d'une architecture modulaire (« modular monolith ») : pas de microservices, un seul artefact déployable, mais un découpage par bounded context.

Organisation frontend/backend. Séparation physique complète : **backend/** (Java/Maven) et **frontend/** (Vue/Vite), chacun avec son propre **Dockerfile**. La communication se fait **uniquement par HTTP/JSON** via l'API REST **/api/v1**.

Communication entre composants.

- Le frontend appelle l'API via **Axios**, base URL `/api/v1` en production (`frontend/src/services/api.js`).
- En production, frontend et API sont **sur le même domaine** : Nginx sert le SPA et proxifie `/api/` vers `http://backend:8080` (`nginx/nginx.vps.conf`) — pas de problème CORS en production, CORS restant configuré côté Spring pour le développement.
- Authentification par **JWT Bearer** transmis dans l'en-tête `Authorization`.

Flux global d'une requête utilisateur (schéma logique) :



Détail d'une requête authentifiée :

1. Le navigateur émet la requête ; Axios y attache `Authorization: Bearer <jwt>` ([api.js](#)).
2. Nginx termine le TLS et proxifie vers le backend (en ajoutant `X-Forwarded-Proto`, exploité par `server.forward-headers-strategy=FRAMEWORK`).
3. `JwtAuthenticationFilter` valide le token et peuple le `SecurityContext` ([auth/data/jwt/JwtAuthenticationFilter.java](#)).
4. `SecurityConfig` autorise/refuse selon le rôle ([config/SecurityConfig.java](#)).
5. Le Controller délègue au Service (logique métier), qui passe par le Repository (accès données).
6. Un Mapper transforme l'entité en DTO ; le Controller emballe le résultat dans `RestResponse<T>` ([shared/response/RestResponse.java](#)).

3. Architecture Backend

Framework. Spring Boot **4.0.6**, Java **21** ([backend/pom.xml](#)). Starters utilisés : `web`, `data-jpa`, `security`, `validation`, `actuator`, `aspectj` (AOP), plus `springdoc-openapi` (Swagger), `jjwt` (JWT) et `micrometer-registry-prometheus`.

Organisation des packages. Découpage **par domaine métier puis par couche** (package-by-feature). Exemple pour le domaine `contrat` :

```
sn.immosn.backend
├── contrat/
│   ├── data/entity/           → Contrat, ContratHistory, StatutContrat, TypeContrat
│   ├── data/repository/       → ContratRepository, ContratHistoryRepository
│   ├── scheduler/             → ContratExpirationJob
│   └── service/ + service/impl/ → ContratService (interface) +
ContratServiceImpl
├── client/web/contrat/
│   ├── controller/           → ContratController
│   ├── dto/                  → ContratCreateRequestDto, ContratResponseDto, ...
│   └── mapper/                → ContratMapper
├── config/                    → SecurityConfig, OpenApiConfig, WebMvcConfig,
RestTemplateConfig
└── shared/                    → exception/, response/, infrastructure/, mapper/,
service/, util/
```

Cette convention est appliquée de façon homogène : la **couche web** (controllers, DTO, mappers) est isolée sous `client.web.<domaine>`, tandis que la **logique métier et données** vit sous `<domaine>` (entity, repository, service). Le package `shared` regroupe tout le transverse.

Couches identifiées et rôle de chacune.

Couche	Emplacement	Rôle
Controller	<code>client.web.<domaine>.controller</code>	Point d'entrée HTTP. Valide les entrées (<code>@Valid</code>), applique l'autorisation fine (<code>@PreAuthorize</code>), délègue au service, emballe la réponse dans <code>RestResponse</code> .

Couche	Emplacement	Rôle
		Ne contient pas de logique métier. Ex. AuthController.java , ContratController.java .
Service	<code><domaine>.service</code> (interface) + <code>service/impl</code>	Logique métier, règles de gestion, transactions (<code>@Transactional</code>), orchestration multi-repository. Ex. ContratServiceImpl.java , AuthService.java .
Repository	<code><domaine>.data.repository</code>	Accès aux données via Spring Data JPA. Interfaces étendant <code>JpaRepository</code> , requêtes dérivées et <code>Specification</code> pour la recherche dynamique. Ex. ContratRepository , AnnonceSpecification .
Entity / Model	<code><domaine>.data.entity</code>	Modèle persistant JPA (<code>@Entity</code>). Ex. User.java , Contrat.java , Annonce.java .
DTO	<code>client.web.<domaine>.dto</code>	Objets de transfert (souvent des <code>record</code> Java) séparant l'API du modèle interne : <code>*RequestDto</code> (entrée, annotés validation) et <code>*ResponseDto</code> (sortie).
Mapper	<code>client.web.<domaine>.mapper</code>	Conversion Entity $\rightleftharpoons$ DTO, implémentée à la main en <code>@Component</code> . Ex. ContratMapper.java , AuthMapper .

Le découpage **interface/implémentation** des services (ex. [ContratService](#) / [ContratServiceImpl](#)) facilite le mock dans les tests et le respect de l'inversion de dépendances.

4. Patterns et principes utilisés

Pour chaque pattern : **où** (citation), **rôle**, **pourquoi utile**.

Layered Architecture / MVC. Découpage Controller → Service → Repository → Entity décrit en §3. *Pourquoi* : séparation des responsabilités, testabilité, maintenabilité. La partie « View » du MVC est déportée dans le SPA Vue.

Dependency Injection (Spring IoC). Injection par constructeur partout, principalement via `@RequiredArgsConstructor` de Lombok (champs `final` injectés). Ex. [ContratServiceImpl](#) reçoit 8 dépendances par constructeur ([ContratServiceImpl.java](#)). *Pourquoi* : couplage faible, dépendances explicites, mockabilité.

Repository Pattern. Spring Data JPA : interfaces de repository sans implémentation manuelle (générée par Spring). *Où* : tous les `*Repository`. *Pourquoi* : abstrait l'accès aux données, requêtes dérivées du nom de méthode, pagination native.

Service Layer Pattern. Toute la logique métier est confinée dans la couche service, exposée via interfaces ([ContratService](#), [AnnonceService](#), [LeadService](#)...). *Pourquoi* : centralise les règles de gestion et les frontières transactionnelles.

DTO Pattern. Entrées/sorties API découplées des entités via des `record` ([ContratCreateRequestDto](#), [AuthResponseDto](#)...). *Pourquoi* : évite d'exposer le modèle interne, permet la validation d'entrée, contrôle précis du contrat d'API.

Mapper Pattern. Conversion explicite Entity → DTO dans des composants dédiés ([ContratMapper.java](#)).

Pourquoi : isole la logique de transformation, gère les cas nuls (ex. annonce/lead/visite optionnels).

Builder Pattern. Lombok [@Builder](#) sur les entités ([Contrat](#), [Annonce](#), [UserSession](#)). Utilisé intensivement pour construire des [Contrat](#) selon le type VENTE/LOCATION ([ContratServiceImpl.java](#)). *Pourquoi* : construction lisible d'objets à nombreux champs optionnels.

State Machine Pattern (machine à états). Les transitions de statut des contrats sont validées contre une table de transitions autorisées ([TRANSITIONS_AUTORISEES](#), [Map<StatutContrat, Set<StatutContrat>>](#)) ([ContratServiceImpl.java](#)). Toute transition passe par [validateTransition\(\)](#). *Pourquoi* : garantit la cohérence du cycle de vie (EN_ATTENTE → ACTIF → EXPIRE/RESILIE...) et empêche les changements d'état illégaux. **C'est l'un des points techniques les plus aboutis du projet.**

Scheduled Job / Tâche planifiée. [@Scheduled\(cron = ...\)](#) pour expirer automatiquement les contrats échus ([contrat/scheduler/ContratExpirationJob.java](#)) ; nettoyage des tokens blacklistés ([BlacklistedTokenCleanupJob](#)) ; heartbeat Neon ([NeonHeartbeat](#)). *Pourquoi* : automatise les tâches récurrentes sans intervention.

Aspect-Oriented Programming (AOP). [DatabaseRetryAspect](#) intercepte tous les [@Service](#) et retente les opérations en cas de connexion DB perdue (cold start Neon), avec backoff 2s/4s/6s ([shared/infrastructure/DatabaseRetryAspect.java](#)). *Pourquoi* : résilience transparente face au serverless Neon, sans polluer le code métier.

Filtre / Chain of Responsibility (Servlet Filter). [JwtAuthenticationFilter](#) extends [OncePerRequestFilter](#), inséré avant [UsernamePasswordAuthenticationFilter](#) ([JwtAuthenticationFilter.java](#)). *Pourquoi* : authentifie chaque requête de façon centralisée.

Singleton. Tous les beans Spring ([@Service](#), [@Component](#), [@Configuration](#), [@RestControllerAdvice](#)) sont des singletons gérés par le conteneur IoC. *Pourquoi* : instance unique partagée, sans gestion manuelle du cycle de vie.

CommandLineRunner (initialisation / seed). [RoleDataInitializer](#) implements [CommandLineRunner](#) crée les rôles et le SUPER_ADMIN au démarrage ([RoleDataInitializer.java](#)). *Pourquoi* : garantit un état initial cohérent à chaque démarrage.

Specification Pattern. [AnnonceSpecification](#) construit des prédicats JPA dynamiques pour la recherche multi-critères d'annonces. *Pourquoi* : compose des filtres optionnels (prix, région, type...) sans multiplier les méthodes de repository.

Côté frontend : Store/State pattern (Pinia), Service layer (un module Axios par domaine), Interceptor pattern (intercepteurs Axios requête/réponse) — détaillés en §9.

5. Workflow métier complet

5.1 Inscription / création d'un compte CLIENT

```
Frontend (InscriptionView.vue)
  ↓ validation formulaire (champs requis, format)
Axios POST /api/v1/auth/register          (authService.register)
```

```

↓
AuthController.register (@Valid AuthRegisterRequestDto) [public]
↓
AuthService.register (@Transactional)
  • vérifie unicité email → EntityExistException (409) sinon
  • hash BCrypt du mot de passe
  • attribue le rôle CLIENT
  • génère un JWT + crée une UserSession
↓
UserRepository.save → PostgreSQL
↓
AuthMapper.toAuthResponseDto → RestResponse<AuthResponseDto> (201, token inclus)
↓
Frontend : authStore stocke le token (localStorage) + header Axios

```

Source : [AuthController.java](#), [AuthService.java](#).

5.2 Authentification (login)

`POST /api/v1/auth/login` → `AuthService.login` authentifie via `AuthenticationManager` (provider `DaoAuthenticationProvider` + `BCryptPasswordEncoder`), met à jour `dernierConnexion`, génère un JWT, crée une session ([AuthService.java](#)). Le frontend (`authStore.login`) decode le rôle depuis le JWT, le **sanitise** contre la liste blanche `ALLOWED_ROLES`, et redirige selon le rôle ([stores/authStore.js](#)).

5.3 Autorisation et gestion des rôles

Deux niveaux complémentaires :

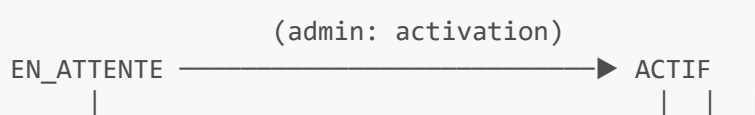
1. **Au niveau URL** (`SecurityConfig`) : `requestMatchers(...).hasRole(...)` / `.hasAnyRole(...)` / `.permitAll()` / `.authenticated()` — modèle **deny-by-default** (`anyRequest().authenticated()`) ([SecurityConfig.java](#)).
2. **Au niveau méthode** (`@EnableMethodSecurity` + `@PreAuthorize`) : 38 occurrences de `@PreAuthorize` dans les controllers pour une granularité fine (ex. gestion des admins réservée à `hasRole('SUPER_ADMIN')`).

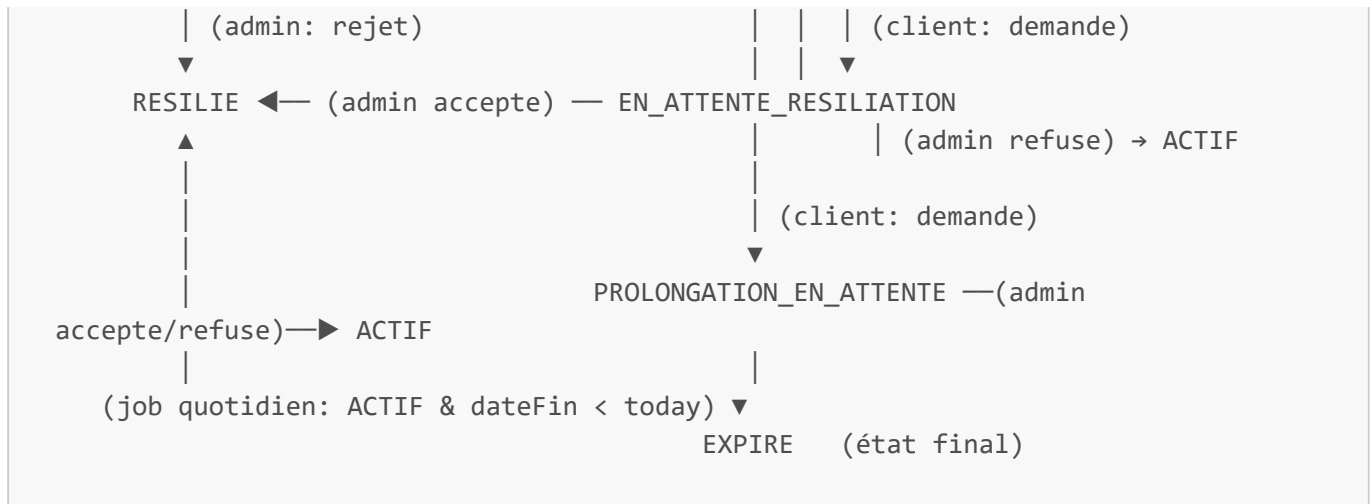
5.4 CRUD type — Annonces

`AnnonceController` expose la création (`POST`, ADMIN), la lecture publique (`GET /annonces`, `GET /annonces/{id}`), la recherche (`POST /annonces/search` via `AnnonceSpecification`), la modification et l'archivage logique (`isArchived`). Le service vérifie notamment qu'une annonce liée à un contrat ne peut pas être supprimée brutalement (dépendance `ContratRepository` injectée — visible dans le test [AnnonceServiceTest.java](#)).

5.5 Workflow spécifique — Cycle de vie d'un contrat (machine à états)

C'est le workflow le plus riche. États (`StatutContrat`) et transitions autorisées ([ContratServiceImpl.java](#)) :





Chaîne d'un exemple — **demande de résiliation par le client** :

```

Frontend (ContratDetailView.vue) → Axios PUT /api/v1/contrats/{id}/resiliation
↓
ContratController (@PreAuthorize client propriétaire)
↓
ContratServiceImpl.demanderResiliation
  • loadClientContrat() : vérifie que le contrat appartient bien au client
  • validateTransition(ACTIF → EN_ATTENTE_RESILIATION) ← machine à états
  • set statut + motifResiliation
↓
ContratRepository.save
↓
ContratHistoryService.record(...) ← trace l'historique
↓
ContratMapper.toDto → RestResponse<ContratResponseDto>
  
```

Source : [ContratServiceImpl.java](#). Particularités métier remarquables : montant et **dateFin** d'un contrat LOCATION sont des **valeurs dérivées** (loyer × durée ; dateDebut + durée) protégées en écriture directe ([ibid. L243-L262](#)) ; création possible automatiquement depuis une visite clôturée ([createFromVisite](#)).

5.6 Workflow visite → lead → contrat

La création d'un contrat à partir d'un lead lié à une visite est **forcée de passer par la clôture de visite** ([cloturerVisite\(AVEC_CONTRAT\)](#)), pour garantir la cohérence transactionnelle visite → lead → contrat dans une seule transaction ([ContratServiceImpl.java](#)). La conversion positionne le lead en **CONVERTI** avec historisation.

5.7 Gestion des administrateurs (SUPER_ADMIN)

Création ([POST /auth/admin](#)), liste ([GET /auth/admins](#)), modification ([PUT /auth/admins/{id}](#)), archivage/restauration ([PATCH /auth/admins/{id}/archive|restore](#)) — tous réservés à **SUPER_ADMIN** ([SecurityConfig.java](#)). Côté UI, l'action « Révoquer l'accès administrateur » s'appuie sur la logique d'archivage (réversible via « Restaurer »).

6. Sécurité

Authentification. JWT stateless (HMAC-SHA, librairie JJWT 0.11.5). `SessionCreationPolicy.STATELESS` : aucune session serveur (`SecurityConfig.java`). Le token porte le `subject` (email) et un claim `roles` (`JwtTokenProvider.java`). Validation à chaque requête par `JwtAuthenticationFilter`.

Invalidation de token (logout). À la déconnexion, le token est ajouté à une **liste noire** (`BlacklistedToken`) et la session marquée inactive (`AuthService.java`). `validateToken()` rejette tout token blacklisté (`JwtTokenProvider.java`). Un job nettoie périodiquement les tokens expirés (`BlacklistedTokenCleanupJob`).

Autorisation / RBAC. Modèle **deny-by-default** à deux niveaux (URL + méthode) décrit en §5.3. Les rôles sont matérialisés comme autorités `ROLE_<NOM>` (`User.java`).

Hachage des mots de passe. `BCryptPasswordEncoder` (`SecurityConfig.java`) ; aucun mot de passe stocké ou journalisé en clair.

Désactivation de compte. L'entité `User` implémente `UserDetails` ; le flag `isArchived` désactive immédiatement le compte (`isEnabled()`, `isAccountNonLocked()` renvoient `!isArchived`) (`User.java`) : un compte archivé ne peut plus s'authentifier.

Protection des endpoints. Whitelist explicite des routes publiques (`/auth/register`, `/auth/login`, `GET /annonces`, géocodage...), tout le reste authentifié. Swagger et la console H2 **désactivés en production** (`application-neon.properties`) et bloqués par Nginx (`/actuator`, `/h2-console` → 403) (`nginx.vps.conf`).

Validation des entrées. Bean Validation (`spring-boot-starter-validation`) : DTO d'entrée annotés et validés par `@Valid` au niveau controller ; les erreurs sont agrégées champ par champ par le `GlobalExceptionHandler` (voir §7).

CORS. Configuration centralisée : origines autorisées paramétrables, méthodes et en-têtes restreints, `allowCredentials` activé, appliqué à `/api/**` (`SecurityConfig.java`). En production, origines limitées à `https://immosn.cloud` (`application-neon.properties`).

Durcissement réseau / transport (Nginx). HTTPS forcé (redirection 80→443), TLS 1.2/1.3, **HSTS preload**, en-têtes `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy` et une **Content-Security-Policy** détaillée (`nginx.vps.conf`). **Rate limiting** différencié : 10 req/min sur `/auth/login|register`, 30 req/min sur le reste de l'API, réponse 429 (*ibid.* L32-L33, L96-L111). Le backend n'est publié **que sur le loopback** de l'hôte (`127.0.0.1:8080`), inaccessible directement depuis l'extérieur (`docker-compose.yml`).

Sécurité côté frontend. Le rôle issu du JWT est **sanitizé** contre une liste blanche pour empêcher l'injection d'une valeur arbitraire, et les tokens expirés sont purgés du `localStorage` (`authStore.js`). Sur réponse 401, l'intercepteur Axios purge le token et redirige vers la connexion (`api.js`).

Remarque honnête (limite). Le JWT et le rôle sont stockés en `localStorage`, ce qui les expose à un vol en cas de faille XSS. C'est un compromis classique pour un SPA ; une alternative plus stricte serait un cookie `HttpOnly`. Le projet en atténue partiellement le risque par une CSP stricte côté Nginx. Voir §14.

7. Gestion des erreurs

Stratégie globale. Un **gestionnaire d'exceptions centralisé** `@RestControllerAdvice`

(`shared/exception/GlobalExceptionHandler.java`) traduit chaque type d'exception en code HTTP et message cohérents :

Exception	HTTP	Cas
<code>EntityNotFoundException</code>	404	Entité introuvable
<code>EntityExistException</code>	409	Conflit (email déjà pris...)
<code>IllegalStateException</code>	422	Règle métier violée (ex. transition de statut illégale)
<code>IllegalArgumentException</code>	400	Argument invalide
<code>AccessDeniedException</code>	403	Permissions insuffisantes
<code>AuthenticationException</code>	401	Token absent/invalide
<code>MethodArgumentNotValidException</code>	400	Échec de validation <code>@Valid</code> (erreurs par champ)
<code>HttpMessageNotReadableException</code> / <code>MissingServletRequestParameterException</code> / <code>MethodArgumentTypeMismatchException</code>	400	Corps/paramètre malformé
<code>DataAccessException</code>	503	Base indisponible (cold start Neon)
<code>Exception</code> (fallback)	500	Erreur inattendue

Réponses API uniformes. Toutes les réponses (succès comme erreur) utilisent l'enveloppe `record RestResponse<T> : { success, status, message, data, timestamp }` (`RestResponse.java`). Les listes paginées utilisent `PagedResponse`. *Bénéfice* : le frontend a un contrat de réponse prévisible (`response.data.data`, `response.data.message`).

Messages utilisateur. Les messages d'erreur backend sont en français et orientés utilisateur. Côté frontend, l'intercepteur Axios enrichit l'erreur (`error.userMessage`) et un utilitaire `getErrorMessage` extrait les messages détaillés de validation pour les toasts (`utils/messages.js`).

Logs. Niveaux de log adaptés à la gravité dans le handler (`debug` pour les erreurs attendues type 404/validation, `warn` pour les violations de règle, `error` + stack trace pour les 500/503). Stratégie de logging détaillée en §10.

Note sur la sécurité des erreurs. Le handler générique 500 ne renvoie **jamais** le détail technique au client (« Une erreur interne est survenue ») mais log la stack complète côté serveur — évite la fuite d'informations.

8. Base de données

SGBD. PostgreSQL managé via **Neon** (serverless) en production ([application-neon.properties](#)) ; **H2 en mémoire** pour le développement et les tests ([application.properties](#)). Les deux drivers sont présents au `pom.xml`.

ORM. JPA / Hibernate. `ddl-auto` configurable (`update` en dev, `validate` recommandé en prod via `DDL_AUTO`). `open-in-view=false` en production (bonne pratique : pas de session JPA ouverte pendant le rendu) ([application-neon.properties](#)).

Pool de connexions — HikariCP calibré pour Neon serverless. Configuration soignée et **documentée** pour absorber les *cold starts* et éviter les connexions mortes : `keepalive 30s`, `max-lifetime 4 min` (< seuil Neon ~5 min), `initialization-fail-timeout=-1`, `connection-test-query=SELECT 1` ([application-neon.properties](#)). Complété par trois mécanismes applicatifs : `NeonWarmup` (au démarrage), `NeonHeartbeat` (`@Scheduled 4 min`) et `DatabaseRetryAspect` (retry AOP).

Modèle de données — principales entités et relations.



Cardinalités et contraintes notables :

- User** ↔ **Role** : **Many-to-Many** via table de jointure `user_roles` (`@ManyToMany(fetch = EAGER)`) ([User.java](#)).
- Contrat** → **User/Annonce** : **Many-to-One obligatoires** (`optional = false, nullable = false`) ; → **Lead** **Many-to-One optionnel** ([Contrat.java](#)).
- Annonce** → **AnnonceCommodite** : **One-to-Many** avec `cascade = ALL + orphanRemoval` ; clés composites (`AnnonceCommoditeId`, `AnnonceFavorisId`) via `@EmbeddedId` ([Annonce.java](#)).
- Contraintes d'intégrité : `email unique + not null` sur `users` ; champs monétaires en `BigDecimal` avec `precision/scale` (12,2 pour les contrats, 10,2 pour les annonces) — précision financière correcte.
- Énumérations persistées en `@Enumerated(EnumType.STRING)` (statuts contrats/leads/visites) — lisibles en base, robustes au réordonnement.
- Horodatage automatique via callbacks `@PrePersist/@PreUpdate` (`createdAt/updatedAt`) sur les entités principales.

Migrations. Pas d'outil de migration versionnée (Flyway/Liquibase **absent** du `pom.xml`). Le schéma est géré par Hibernate `ddl-auto` (`update/validate`). C'est un point d'amélioration identifié en §14.

9. Frontend

Framework. **Vue.js 3** (Composition API, `<script setup>`), build **Vite 8**, langage JavaScript (`frontend/package.json`). Dépendances clés : `vue-router` (routing), `pinia` (state), `axios` (HTTP), `leaflet` + `@vue-leaflet/vue-leaflet` (cartes), `lucide-vue-next` (icônes).

Architecture des composants. Organisation par responsabilité :

```
frontend/src/
├─ views/           → 35 vues (pages), réparties client (annonces, contrats,
visites, favoris, profil...)
│                   et admin/ (23 vues : dashboard, gestion annonces, leads,
contrats, admins...)
├─ components/      → 18 composants réutilisables (+ sous-dossier admin/)
├─ layouts/         → ClientLayout.vue, AdminLayout.vue (chrome distinct par
interface)
├─ services/        → 15 modules d'appel API (un par domaine)
├─ stores/          → 3 stores Pinia (auth, favoris, toast)
├─ composables/     → usePagination.js (logique réutilisable)
└─ router/          → index.js (routes + guard global)
```

Gestion d'état (Pinia). Trois stores : `authStore` (token, rôle, login/logout/profil), `favorisStore`, `toastStore` (notifications). `authStore` est défini en `setup store` (Composition API) et persiste le token/rôle dans `localStorage` (`stores/authStore.js`).

Services API. Une couche service par domaine (`authService`, `contratService`, `annonceService...`), toutes construites sur une **instance Axios partagée** (`services/api.js`) avec `baseUrl`, timeout 15 s et intercepteurs (`api.js`). *Bénéfice* : centralisation de l'auth (header Bearer), de la gestion FormData (upload) et des erreurs.

Routing. `vue-router` en mode history, **lazy-loading** systématique des vues (`() => import(...)`) pour le découpage du bundle. Routes organisées en deux arbres avec layouts distincts (client / et admin /`admin`) (`router/index.js`).

Guards / protections. Un **guard global** `beforeEach` applique le contrôle d'accès côté client à partir des métadonnées de route (`meta.requiresAuth`, `meta.role`) : redirection vers la connexion si non authentifié, cloisonnement client/admin (`router/index.js`). Ce contrôle est **défensif** (UX) ; l'autorisation réelle reste imposée par le backend.

Validation des formulaires. Validation côté client dans les vues (champs requis, formats) avant appel API, complétée par la validation backend (Bean Validation) qui fait foi. Composants d'entrée dédiés (`InputField.vue`, `PhoneInput.vue`).

Réutilisation des composants. Bibliothèque de composants transverses : `StatusBadge`, `Badge`, `AnnonceCard`, `ImageGallery`, `LocationMap`, `NavBar`, `AppFooter`, `ToastContainer`, et côté admin

`ConfirmModal`, `StatsCard`, `FormStepper`, `ToastNotification`. Le composable `usePagination` factorise la pagination entre vues.

Qualité frontend. Double *linting* `oxlint` + `ESLint` (config plate `eslint.config.js`, plugin Vue) et `Prettier`, orchestrés par les scripts npm (`package.json`).

10. Monitoring et qualité

⚠ **Précision factuelle importante.** Le sujet mentionne Prometheus, Grafana et SonarQube. L'audit du dépôt montre que :

- **SonarCloud** (service cloud) **est** intégré — pas un SonarQube auto-hébergé.
- L'application **expose** des métriques Prometheus, mais **aucun service Prometheus ni Grafana n'est défini dans le dépôt** (ni dans `docker-compose.yml`, ni de fichier de configuration). Le scraping est assuré par un Prometheus **externe** (systemd sur l'hôte), d'après les commentaires de configuration. Je documente donc ce qui est réellement présent.

Exposition des métriques (Micrometer / Prometheus).

- *Rôle* : exposer des métriques applicatives (latences HTTP, JVM, pool...) au format Prometheus.
- *Intégration* : dépendance `micrometer-registry-prometheus` (`pom.xml`) ; endpoint `/actuator/prometheus` exposé et **rendu public uniquement sur le loopback** ; histogrammes p95/p99 activés sur `http.server.requests` et tag commun `application` pour le filtrage (`application-neon.properties`). Seuls `health` et `prometheus` sont exposés en production (aucun endpoint Actuator sensible) ; Nginx bloque `/actuator` côté public.
- *Bénéfice* : observabilité des performances et de la santé applicative sans instrumentation manuelle.

Prometheus / Grafana (externes). D'après les commentaires de `docker-compose.yml`, un Prometheus hôte scrape `http://127.0.0.1:8080/actuator/prometheus`. La stack de visualisation (Grafana) n'est **pas versionnée dans ce dépôt** — elle relève de l'infrastructure du VPS. *À mentionner tel quel en soutenance* : l'application est **prête pour** Prometheus/Grafana ; leur déploiement est externe.

Healthcheck. `/actuator/health` (sans dépendance DB pour éviter les faux négatifs liés à Neon) sert au healthcheck Docker (`docker-compose.yml`) et au gating du déploiement (§12).

SonarCloud (qualité de code).

- *Rôle* : analyse statique (bugs, code smells, vulnérabilités, couverture, duplication).
- *Intégration* : configuration `sonar-project.properties` (projectKey, sources `backend/src/main` + `frontend/src`, rapport JaCoCo) (`sonar-project.properties`) ; lancé dans la CI sur chaque PR via `mvn sonar:sonar` vers `sonarcloud.io` (`.github/workflows/ci.yml`).
- *Bénéfice* : barrière qualité automatique à chaque pull request, décoration des PR, suivi de la dette technique.

Couverture (JaCoCo). Le plugin JaCoCo génère un rapport `jacoco.xml` exploité par SonarCloud et archivé comme artefact CI (`pom.xml`, `ci.yml`).

11. Tests

Types de tests présents. Tests **unitaires** backend (JUnit 5 + Mockito + AssertJ), plus un test de **contexte** Spring (`BackendApplicationTests`) et un test de **controller web** (`AuthControllerTest`, avec `spring-security-test`). Outils fournis par `spring-boot-starter-test` et `spring-security-test` (`pom.xml`).

Inventaire (≈ 22 méthodes `@Test` sur 5 classes) :

Classe de test	Méthodes <code>@Test</code>	Nature
<code>AnnonceServiceTest</code>	10	Unitaire (service mocké)
<code>FavorisServiceTest</code>	6	Unitaire
<code>AuthControllerTest</code>	4	Web / controller
<code>DashboardServiceTest</code>	1	Unitaire
<code>BackendApplicationTests</code>	1	Démarrage du contexte

Stratégie. Les tests de service suivent le pattern **mock & verify** : dépendances `@Mock`, classe sous test `@InjectMocks`, vérification du comportement et des arguments via `ArgumentCaptor` (`AnnonceServiceTest.java`). Lisibilité renforcée par `@DisplayName`.

Exécution & couverture. `mvn test` lancé en CI sur chaque PR (`ci.yml`) ; couverture mesurée par JaCoCo et remontée à SonarCloud.

Limite honnête. La couverture est **partielle** : le domaine le plus complexe (machine à états des contrats, `ContratServiceImpl`) et le frontend n'ont pas de tests automatisés versionnés. C'est l'axe d'amélioration prioritaire (§14). Un dossier `backend/testApi` existe (tests d'API manuels/exploratoires).

12. Déploiement

Environnement. **VPS** (serveur dédié) sous Docker, derrière Nginx, base PostgreSQL **Neon** managée externe. Domaine `immosn.cloud` avec certificats **Let's Encrypt** montés en lecture seule (`docker-compose.yml`).

Conteneurisation (Docker). `docker-compose.yml` orchestre **deux services** sur un réseau bridge applicatif :

- `backend` (image construite depuis `backend/Dockerfile`), profil Spring `neon`, publié sur `127.0.0.1:8080`, healthcheck Actuator, logs persistés en volume ;
- `nginx` (build multi-stage : compile le frontend Vite puis sert le statique + reverse proxy), ports 80/443, démarrage conditionné à la **santé** du backend (`depends_on: condition: service_healthy`).

Plusieurs variantes de compose existent : `docker-compose.yml` (production VPS), `docker-compose.local.yml` et `docker-compose.dev.yml` (local/dev).

Configuration & variables d'environnement. Externalisées via `.env` (template `.env.example`). Principales : `JWT_SECRET`, `JWT_EXPIRATION`, `DB_HOST/NAME/USER/PASSWORD`, `DB_POOL_SIZE`, `DDL_AUTO`, `CORS_ORIGINS`, `SUPER_ADMIN_*`, `CONTRAT_EXPIRATION_CRON`, `VITE_API_URL`, `VITE_CLOUDINARY_*` (`.env.example`, `docker-compose.yml`). Le profil `neon` est activé par `SPRING_PROFILES_ACTIVE`. **Aucun secret n'est commité** (template uniquement).

Pipeline CI/CD (GitHub Actions). Deux workflows :

1. **CI** — **ci.yml** (sur *pull request* vers **master**) : checkout (historique complet pour SonarCloud) → JDK 21 (cache Maven) → **mvn test** → upload du rapport JaCoCo → analyse SonarCloud ([.github/workflows/ci.yml](#)). Concurrence annulant les runs obsolètes.
2. **Deploy** — **deploy.yml** (sur *push* vers **master**) : connexion **SSH** durcie (clé ed25519, **StrictHostKeyChecking=yes**, **known_hosts** vérifié) → sur le VPS : **git pull** → **docker compose up -d --build** → **healthcheck gating** : attente jusqu'à 4 min de l'état **healthy** du conteneur backend ; si échec, dump des logs et **sortie en erreur** (déploiement considéré comme échoué) ; sinon **docker image prune** ([.github/workflows/deploy.yml](#)). Concurrence **cancel-in-progress: false** pour ne pas interrompre un déploiement en cours.

Workflow complet :

```

Développement (branche)
  ↓ commit / Pull Request → master
CI : Build + Tests (mvn test) + JaCoCo
  ↓
Analyse qualité (SonarCloud)          ← barrière qualité PR
  ↓ merge → push master
Deploy : SSH VPS → git pull → docker compose up -d --build
  ↓
Healthcheck gating (/actuator/health, max 4 min)
  ↓ (succès)                          (échec → logs + exit 1)
Production en ligne (Nginx + backend)
  ↓
Monitoring (/actuator/prometheus scrappé par Prometheus hôte)

```

13. Points forts techniques du projet

Bonnes pratiques.

- Découpage **package-by-feature** cohérent et homogène sur tout le backend ; séparation nette couche web / métier / données.
- **Interfaces de service** systématiques (**X / XImpl**) → faible couplage, testabilité.
- **Enveloppe de réponse uniforme** (**RestResponse/PagedResponse**) et **gestion d'erreurs centralisée** exhaustive (11 handlers couvrant 400→503).
- DTO en **record**, validation déclarative, mappers explicites.
- Configuration **multi-profil** (H2 dev / Neon prod) propre, secrets externalisés, jamais commités.

Choix d'architecture.

- **Machine à états** explicite pour les contrats : règle métier centralisée, transitions illégales impossibles, valeurs dérivées (LOCATION) protégées en écriture — niveau de rigueur remarquable.
- **Historisation** systématique (***History**) → auditabilité du cycle de vie métier.
- Résilience **Neon serverless** pensée de bout en bout (HikariCP calibré + warmup + heartbeat + retry AOP).

Sécurité.

- **Deny-by-default** à deux niveaux (URL + méthode), JWT stateless avec **blacklist** de tokens, BCrypt, désactivation immédiate via **isArchived**.
- Durcissement **Nginx** complet : HTTPS/HSTS, CSP, en-têtes de sécurité, **rate limiting** différencié, backend exposé seulement sur loopback, Swagger/H2/Actuator fermés en prod.

Maintenabilité. Code abondamment commenté (décisions techniques explicitées), conventions constantes, linting frontend (oxlint+ESLint+Prettier), SonarCloud sur chaque PR, logs structurés multi-fichiers avec rotation.

Scalabilité. Backend **stateless** (sessions JWT, pas d'état serveur) → horizontalement scalable derrière un load balancer ; base managée (Neon) ; frontend statique cacheable (cache 1 an sur les assets immuables, gzip).

14. Limites et améliorations possibles

Améliorations techniques.

1. **Migrations de schéma versionnées** : introduire **Flyway** ou **Liquibase** pour remplacer **ddl-auto** et fiabiliser les évolutions de schéma en production (reproductibilité, rollback).
2. **Couverture de tests** : ajouter des tests sur **ContratServiceImpl** (machine à états — chaque transition autorisée/interdite), des tests d'intégration (**@SpringBootTest** + Testcontainers PostgreSQL), et des tests frontend (Vitest + Vue Test Utils, voire Playwright pour l'E2E).
3. **Stockage du JWT** : envisager un cookie **HttpOnly/SameSite** plutôt que **localStorage** pour réduire la surface XSS (compromis à arbitrer selon les contraintes du SPA).
4. **Refresh token** : le modèle actuel repose sur un access token de longue durée (24 h). Introduire des access tokens courts + refresh tokens améliorerait la sécurité (révocation plus fine).

Optimisations. 5. **Stack de monitoring versionnée** : intégrer Prometheus + Grafana (et éventuellement des dashboards) au dépôt/compose pour rendre l'observabilité reproductible et documentée, plutôt que dépendante de l'hôte. 6. **Pagination/fetch JPA** : auditer les associations **EAGER** (ex. **User.roles**) et les requêtes des mappers pour prévenir d'éventuels problèmes N+1 sur les listes volumineuses. 7. **Cache** : un cache (Caffeine/Redis) sur les référentiels peu mutables (types de bien, commodités) réduirait la charge DB — pertinent avec Neon serverless.

Évolutions fonctionnelles. 8. **Notifications** (email/SMS) sur les événements clés (validation de contrat, acceptation de visite) — la messagerie interne existe déjà comme socle. 9. **Tests de charge / SLO** : formaliser des objectifs de latence en exploitant les histogrammes p95/p99 déjà exposés. 10. **Documentation d'API publiée** : Swagger est désactivé en prod (bon choix sécurité) ; publier une spec OpenAPI statique versionnée pour les intégrateurs.

Rapport généré à partir d'un audit du code source. Chaque section renvoie aux fichiers et lignes correspondants ; aucune fonctionnalité n'a été extrapolée au-delà de ce que le dépôt contient. Les points où le sujet et le dépôt divergent (Prometheus/Grafana non versionnés, SonarCloud vs SonarQube, absence de migrations) sont signalés explicitement.